

UNIVERSIDADE DO MINHO

Licenciatura em Engenharia Informática

Laboratórios de Informática III

2025/2026

Relatório 1ª Fase Projeto

GRUPO 82

a106800, Eduardo Novo Araújo

a106903, Pedro Afonso de Sousa dos Santos

a107323, Tomás Branco Dias

Novembro 2025

Índice

1	Introdução	1
2	Metodologia utilizada e Descrição do sistema	2
2.1	Arquitetura Geral	2
2.2	Estrutura de dados	3
2.3	Descrição dos módulos	3
2.4	Parsing dos dados	4
3	Implementação das funcionalidades	5
3.1	Validação dos Dados	5
3.2	Queries	5
3.2.1	Query 1	5
3.2.2	Query 2	5
3.2.3	Query 3	6
3.3	Gestão de memória	6
4	Testes	7
4.1	Programa de testes	7
5	Conclusões	8
5.1	Análise Crítica e Resultados alcançados	8
5.2	Limitações	8

Capítulo 1

Introdução

Este relatório aborda o desenvolvimento do trabalho prático da disciplina de Laboratórios de Informática III, implementado em linguagem C, inserido no 2º ano da licenciatura em Engenharia Informática.

Sendo o tema centrado num sistema baseado em aeroportos e voos, a partir do qual exploramos um conjunto de dados relativos a aeroportos, avioes, voos, passageiros, reservas e estatísticas de uso do sistema, carregamos os dados para estruturas de dados e utilizamos essa informação para responder a um conjunto de *queries*. Pretendemos fornecer uma visão geral do contexto, abordagem e métodos adotados, recorrendo à aplicação prática de conceitos como modularização e encapsulamento.

Nesta primeira fase, a estrutura do projeto enfatiza a organização modular. Para garantir a modularidade, o projeto é dividido em diversos módulos, cada um com diferentes responsabilidades, como *parse* de dados, execução de *queries* e gestão de estruturas de dados. Esta abordagem visa facilitar a manutenção e a evolução do sistema ao longo das diferentes fases do projeto.

Aliado a estes, a implementação de estratégias eficientes e o uso de bibliotecas, como a *GLib*, e ferramentas auxiliares, o *Valgrind* para análise de uso de memória.

O desenvolvimento deste trabalho prático inclui a criação de um programa principal que processa dados a partir de arquivos CSV, lidando com validações de dados e gerando respostas para *queries*. Inclui também a criação de um programa de testes e, a par disso, criamos também um programa de testes unitários.

Capítulo 2

Metodologia utilizada e Descrição do sistema

2.1 Arquitetura Geral

Dividimos o projeto em diferentes módulos e à medida que fomos desenvolvendo o código, fomos notando alguns módulos com código repetido e procedemos à generalização dos mesmos.

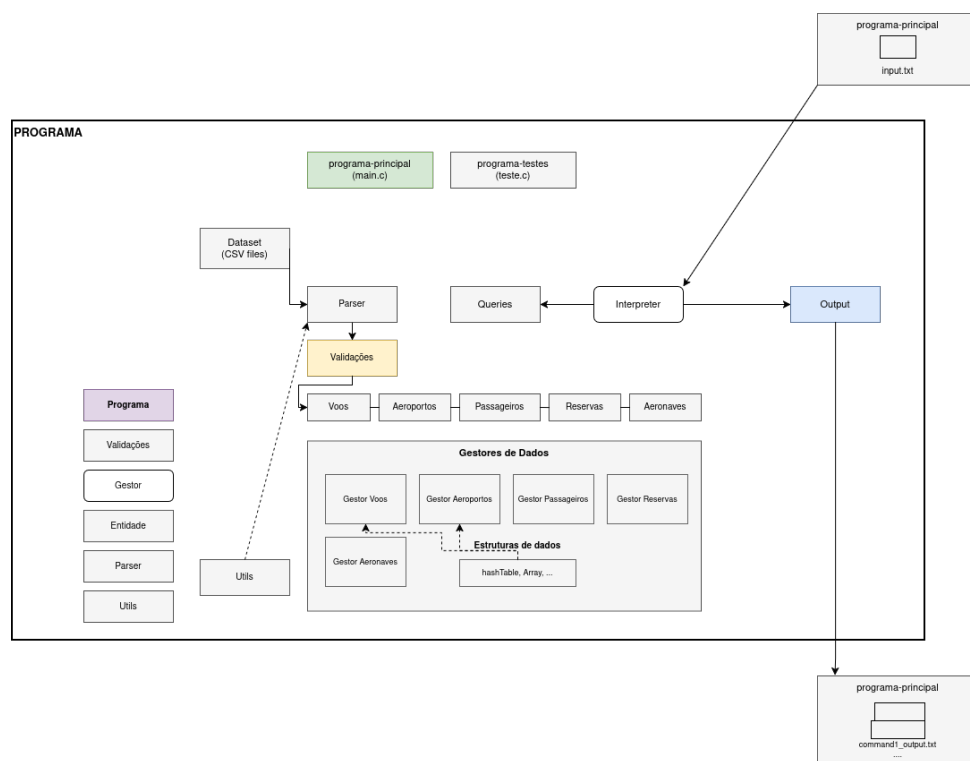


Figura 2.1: Arquitetura de referência para a aplicação a desenvolver

A arquitetura do projeto foi organizada de forma modular, garantindo separação clara de responsabilidades e evitando dependências desnecessárias entre componentes. O programa principal e o programa de testes partilham a mesma base estrutural: ambos iniciam o sistema, chamam o *parser* para carregar o dataset e, de seguida, processam as *queries* através do interpretador. O interpretador é responsável por analisar cada comando e encaminhar a execução para as funções correspondentes, produzindo depois os ficheiros de output.

O carregamento dos dados é realizado por um *parser* genérico, capaz de processar diferentes tipos de

entidades. Cada ficheiro do dataset é lido, validado e convertido para a respetiva estrutura no repositório de dados. As validações formam um bloco autónomo, sendo executadas logo após o parsing, assegurando que apenas dados consistentes são introduzidos no sistema.

Os dados validados são armazenados nos gestores específicos de cada entidade (voos, aeroportos, passageiros, reservas e aeronaves). Estes gestores encapsulam as estruturas internas (hash tables, arrays, etc.) e fornecem operações eficientes de consulta e manipulação. Todo o acesso às entidades é feito através destes módulos, reforçando o encapsulamento e reduzindo o acoplamento entre componentes.

Por fim, o módulo de output é responsável pela geração dos ficheiros finais de resposta às *queries*, produzidos de forma independente da lógica interna de execução. O módulo de utilitários (*utils*) agrega funções transversais utilizadas em vários pontos do sistema, evitando a repetição de código.

2.2 Estrutura de dados

As estruturas de dados foram criadas tendo como base os campos necessários para a resolução das queries e a leitura dos ficheiros CSV. Em vez de reproduzir integralmente toda a informação presente nos datasets, optou-se por armazenar apenas os campos realmente utilizados, reduzindo a memória ocupada e simplificando as operações de acesso.

Cada entidade possui o seu gestor dedicado, que encapsula tanto a representação interna dos dados como as operações associadas. A escolha entre hash tables, arrays e outras estruturas dependeu da natureza das consultas mais frequentes em cada entidade, privilegiando operações rápidas de procura e iteração. Em alguns casos foram criadas estruturas auxiliares para acelerar operações repetitivas, mantendo a lógica do sistema organizada e eficiente.

2.3 Descrição dos módulos

Cada módulo foi criado com uma responsabilidade única e bem delimitada. O *parser* trata exclusivamente da leitura das linhas e do seu mapeamento para objetos; as validações verificam a integridade de cada registo; os gestores de dados armazenam e disponibilizam acesso eficiente às entidades; o interpretador analisa e executa queries; o sistema de output escreve os resultados; e o módulo de utilitários reúne funções de apoio usadas em vários componentes.

A implementação de cada módulo segue a mesma estrutura base: declaração de um tipo opaco no *ficheiro .h*, definição privada da estrutura no *.c*, funções de criação e destruição, métodos de acesso e funções auxiliares associadas. Esta organização garante encapsulamento e facilita a manutenção do código.

Listing 2.1: Arquétipo do módulo.h

```

1
2 #ifndef MODULO_H
3 #define MODULO_H
4
5 // typedef estrutura incompleta para esconder os detalhes da implementacao
6 typedef struct Modulo Modulo;
7
8 // Declaracao da funcao para criar uma nova estrutura do Modulo
9 Modulo* Modulo_novo();
10
11 // Declaracao da funcao para libertar a memoria de uma estrutura Modulo
12 void Modulo_destroi(Modulo* modulo);
13
```

```
14 // Declaracao das funcoes gets e sets para definir um valor no objeto Modulo
15 void Modulo_set...(Modulo* modulo, ...);
16 ... Modulo_get...(Modulo* modulo);
17
18 // Declaracao de eventuais funcoes auxiliares e de execucao que sejam utilizadas por
    outros modulos
19 ... modulo...(...);
20
21 #endif
```

Listing 2.2: Arquétipo do módulo.c

```
1 #include "modulo.h"
2
3 // Definicao da estrutura Modulo, "privada" para o arquivo de implementacao
4 struct Modulo {
5 };
6
7 // Implementacao da funcao para criar uma nova estrutura do Modulo
8 Modulo* Modulo_novo() {
9 };
10
11 // Implementacao da funcao para libertar a memoria de uma estrutura do Modulo
12 void Modulo_destruir(Modulo* modulo) {
13 };
14
15 // Implementacao das funcoes get e set da estrutura do Modulo
16 ... Modulo_get...(Modulo* modulo, ...) {
17 };
18 void Modulo_set...(Modulo* modulo, ...) {
19 };
20
21 //eventuais funcoes de auxiliares e de execucao
22 ... modulo...(...){
23 };
```

2.4 Parsing dos dados

O *parser* utilizado é genérico e parametrizável, permitindo carregar qualquer tipo de entidade através de funções de conversão e inserção fornecidas pelos respectivos módulos. Cada linha do ficheiro é lida, dividida corretamente em colunas (incluindo tratamento de aspas), validada e convertida para o objeto correspondente. Caso ocorra algum erro, a linha original é registada num ficheiro de erros próprio.

A evolução do *parser* ao longo do projeto permitiu eliminar versões duplicadas e concentrar toda a lógica de leitura, validação e criação de objetos numa única implementação reutilizável. Esta abordagem torna o sistema mais simples, mais consistente e mais fácil de estender.

Capítulo 3

Implementação das funcionalidades

3.1 Validação dos Dados

Inicialmente, as validações eram feitas diretamente nos ficheiros de dados, mas com a evolução do projeto e a introdução da modularização, decidimos separar as responsabilidades. As validações sintáticas e lógicas passaram a ser realizadas nos módulos correspondentes a cada entidade. Esta abordagem evita dependências desnecessárias entre entidades e melhora a coerência geral do sistema, tornando o código mais *clean* e fácil de manter.

3.2 Queries

As queries são executadas a partir da *main*, sendo que cada query tem a sua implementação isolada e utiliza os módulos de resultados correspondentes.

3.2.1 Query 1

Listar o resumo de um aeroporto, consoante o identificador recebido por argumento

Para esta query, utilizamos o repositório de aeroportos previamente carregado. O código fornecido pelo utilizador é limpo de espaços e caracteres de nova linha, garantindo que a pesquisa no *HashTable* é precisa. Em seguida, obtém-se a instância do aeroporto e escreve-se a sua informação detalhada num ficheiro de *output*, recorrendo aos métodos de acesso da entidade aeroporto. Caso o aeroporto não exista, é retornada uma linha em branco. Todo este processo é encapsulado na função *query1*, assegurando simplicidade e eficiência no acesso aos dados.

3.2.2 Query 2

Top N aeronaves com mais voos realizados

A query 2 exigiu a criação de uma estrutura auxiliar do tipo *GArray*, onde cada elemento armazena o identificador do avião, fabricante, modelo e número de voos válidos. Inicialmente, esta contagem era feita após o carregamento de todos os aviões e voos, mas posteriormente foi otimizada para ser atualizada à medida que os dados eram carregados a partir dos ficheiros CSV.

Para contar os voos de cada avião, é utilizada uma *GHashTable* que associa o identificador do avião ao número de voos, ignorando os voos cancelados. Para filtrar por fabricante, implementámos a função fabricante *match*, que compara strings de forma case-insensitive, permitindo flexibilidade na pesquisa.

Após processar todos os aviões e preencher o *GArray*, ordenamos os resultados usando a função com um comparador personalizado, garantindo que os aviões com maior número de voos aparecem primeiro.

O *output* é então construído a partir dos N primeiros elementos, libertando-se cuidadosamente toda a memória alocada, incluindo as strings duplicadas e a própria estrutura do *GArray*, prevenindo *memory leaks*. Esta query revelou-se a de maior tempo de execução, devido à necessidade de percorrer todas as entidades e realizar ordenações.

3.2.3 Query 3

Listar o aeroporto com mais partidas entre 2 datas

No contexto do projeto adaptado para aeroportos e voos, a query 3 corresponde a identificar o aeroporto com mais partidas numa determinada faixa temporal. Para isso, utilizámos uma *HashTable* onde a chave é o código do aeroporto e o valor é o número de partidas. Filtramos os voos que estão cancelados e apenas consideramos aqueles cujas datas se situam dentro do intervalo fornecido.

Após contar todas as partidas, os resultados são copiados para um *GArray* de estruturas *ContadorPartidas* para permitir ordenação decrescente pelo número de partidas. Posteriormente, o aeroporto com mais partidas é identificado e as suas informações detalhadas são escritas no ficheiro de *output*. Todo o processo garante eficiência na execução, com a memória a ser libertada corretamente ao final da query.

3.3 Gestão de memória

Durante toda esta primeira fase, houve um cuidado contínuo em evitar *memory leaks*. Foram implementadas funções específicas para libertar estruturas de dados complexas e foi feita a utilização das funções de *free* predefinidas nas bibliotecas GLib. Apesar disso, alguns erros escaparam, exigindo análise com o *Valgrind*, o qual revelou um total de 0,019 MB de *memory leaks* associados à própria biblioteca, o que consideramos aceitável para esta fase do projeto.

Capítulo 4

Testes

4.1 Programa de testes

Foi criado um programa de testes automático para validar cada *query*, comparando ficheiros de output gerados com os esperados e registando discrepâncias linha a linha. Além de verificar exatidão, o programa mede tempo de execução e memória usada, fornecendo estatísticas detalhadas. Este processo permitiu identificar erros e garantir a fiabilidade do sistema antes de prosseguir com o desenvolvimento completo.

Durante a criação do programa de testes, enfrentamos algumas dificuldades, destacando-se exibir a linha específica do código em caso de discrepância nos resultados, mas mais tarde superada.

Capítulo 5

Conclusões

5.1 Análise Crítica e Resultados alcançados

Notamos que o grande desafio deste projeto incidia na programação a grande escala, com base na grande quantidade de dados a analisar, e consideramos ter implementado um código que segue boas práticas de modularidade e encapsulamento. No entanto, o encapsulamento pode não estar completo, porque fomos encapsulando ao longo do desenvolver do projeto, não o priorizando, pois não seria avaliado nesta fase, ao contrário da modularização, onde nos focamos mais.

Desta forma, foi necessário manipular estruturas de dados e algoritmos que correspondessem tanto a nível de complexidade como em eficiência, aos desafios propostos concretizando, assim, os objetivos desta primeira fase.

Assim, consideramos que com o desenvolvimento desta parte do projeto conseguimos explorar e aprimorar os nossos métodos de programação, exploramos e conhecemos uma API nova para nós, *GLib*, assim como diversas ferramentas auxiliares e foi ainda possível consolidar os conceitos de modularidade e encapsulamento.

5.2 Limitações

Uma das limitações que enfrentamos durante o desenvolvimento do projeto foi termos percebido o impacto da modularização numa fase bastante avançada do mesmo, o que nos levou a reestruturar uma grande parte do código. Além disso, tivemos dificuldade em detetar e corrigir *memory leaks* muitos específicos e profundos nas definições de funções.